

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. **Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.**

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

if  $n \leq 0$  then
  | return 0
else
  |  $k = 1$ 
  | for  $i = 1, \dots, n$  do
  |   |  $k = 3 * k$ 
  |   | MYSTERY2( $k$ )
  | return MYSTERY1( $n/3$ ) +  $k + \log k$ 

```

```

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  | return 1
else
  | return MYSTERY2( $n/2$ ) +  $n^2 + n + 1$ 

```

Soluzione. Analizziamo prima il costo di MYSTERY2. Tale funzione richiama se stessa una volta su input $n/2$ e le altre operazioni nella chiamata hanno costo costante. L'equazione di ricorrenza di MYSTERY2 è quindi:

$$T'(n) = \begin{cases} 1 & n \leq 0 \\ T'(n/2) + 1 & n > 0 \end{cases}$$

Tale equazione di ricorrenza può essere risolta con il Master Theorem:

$$\alpha = \log_2 1 = 0 = \beta \Rightarrow T'(n) = \Theta(n^\alpha \log n) = \Theta(\log n)$$

La complessità di MYSTERY1 dipende dal ciclo for e dalla chiamata ricorsiva nell'ultima riga. Il ciclo for viene eseguito n volte. In ogni iterazione il valore k , inizialmente 1, viene triplicato ed utilizzato come input per una chiamata alla funzione MYSTERY2. Poiché la funzione MYSTERY2 ha un costo logaritmico il costo complessivo del ciclo for è:

$$\begin{aligned}\Theta(\log 3) + \Theta(\log 3^2) + \cdots + \Theta(\log 3^n) &= \sum_{i=1}^n \Theta(\log 3^i) = \sum_{i=1}^n \Theta(i \log 3) = \Theta\left(\log 3 \sum_{i=1}^n i\right) = \\ &= \Theta\left(\log 3 \frac{n(n+1)}{2}\right) = \Theta(n^2)\end{aligned}$$

L'equazione di ricorrenza di MYSTERY1 è quindi:

$$T(n) = \begin{cases} 1 & n \leq 0 \\ T(n/3) + n^2 & n > 0 \end{cases}$$

Tale equazione di ricorrenza può essere risolta col il Master Theorem:

$$\alpha = \log_3 1 = 0 < 2 = \beta \Rightarrow T(n) = \Theta(n^\beta) = \Theta(n^2)$$

2. Si consideri un array di interi un array di interi $A = [x_1, \dots, x_n]$ per cui è noto a priori che l'array contiene unicamente $0 \leq N \leq n$ valori distinti. Ad esempio:

$A = [2, 3, 3, 3, 1, 1, 3, 1]$ contiene $n = 8$ interi ma solo $N = 3$ valori distinti, gli interi 1, 2, 3.

Si consideri, inoltre, il seguente algoritmo di ordinamento, REPSORT, che utilizza l'informazione sul numero N di valori distinti per ottimizzare il tempo di calcolo. L'algoritmo riceve in input l'array $A[1, \dots, n]$ e il numero di valori distinti N .

Algorithm 2: REPSORT(INT $A[1, \dots, n]$, INT N)

```

Let  $B[1, \dots, N]$  be a new array //  $B$  stores the  $N$  distinct integers
 $H = \text{HashTab}()$  //  $H$  stores the number of repetitions of each integer
 $k = 1$ 
for  $i = 1, \dots, n$  do
     $cnt = \text{SEARCH}(H, A[i])$ 
    if  $cnt \neq \text{NIL}$  then
        |  $\text{INSERT}(H, A[i], cnt + 1)$  // Increase the count of  $A[i]$ 
    else
        |  $B[k] = A[i]$  // Save  $A[i]$  in  $B$ 
        |  $k = k + 1$ 
        |  $\text{INSERT}(H, A[i], 1)$  // Save  $A[i]$  in  $H$  with starting count 1
MERGESORT( $B, 1, N$ )
 $k = 1$ 
for  $i = 1, \dots, N$  do
     $cnt = \text{SEARCH}(H, B[i])$ 
    for  $j = 1, \dots, cnt$  do
        |  $A[k] = B[i]$ 
        |  $k = k + 1$ 

```

L'obiettivo è confrontare la complessità nel caso medio di REPSORT con quella di MERGESORT, determinando quale dei due algoritmi risulti più efficiente (nel caso medio) nei seguenti scenari:

- a) $N = \Theta(\log n)$
- b) $N = \Theta(\sqrt{n})$
- b) $N = \Theta(n)$

Soluzione.

La complessità nel caso medio (nonché nel caso ottimo e pessimo) di MERGESORT dipende unicamente dalla lunghezza n dell'array, risultando $\Theta(n \log n)$. Per quanto riguarda REPSORT, la complessità media dipende dal costo delle operazioni di SEARCH e INSERT sulla tabella hash H . Nel caso medio tali operazioni richiedono tempo costante $O(1)$. Analizzando il codice:

- Il primo ciclo for ha costo (medio) $\Theta(n)$
- La chiamata a MERGESORT sull'array B di lunghezza N ha costo (ottimo/medio/pessimo) $\Theta(N \log N)$
- I due cicli finali che ricostruiscono A ordinato hanno complessivamente costo (medio) $\Theta(n)$

Mettendo tutto insieme, abbiamo che il costo medio di REPSORT è:

$$\Theta(n) + \Theta(N \log N) + \Theta(n) = \Theta(n + N \log N)$$

- Costo medio di REPSORT: $\Theta(n + \log n \log \log n) = \Theta(n)$. In questo caso REPSORT è più efficiente di MERGESORT.
 - Costo medio di REPSORT: $\Theta(n + \sqrt{n} \log \sqrt{n}) = \Theta(n)$. Anche in questo scenario REPSORT è più efficiente di MERGESORT.
 - Costo medio di REPSORT: $\Theta(n + n \log n) = \Theta(n \log n)$. In questo caso, REPSORT è asintoticamente equivalente a MERGESORT. Tuttavia, nella pratica potrebbe risultare più lento, poiché introduce un overhead aggiuntivo dovuto alla gestione della tabella hash.
3. L'algoritmo basato su programmazione dinamica studiato a lezione per risolvere il problema dello zaino utilizza due strutture dati ausiliarie, una matrice di numeri ed una matrice di booleani. Descrivere brevemente in che modo vengono utilizzate queste due strutture dati e mostrare come vengono riempite dall'algoritmo considerando la seguente istanza del problema dello zaino: capacità dello zaino 10 e 4 possibili oggetti con il seguente vettore dei pesi $p[2, 7, 6, 4]$ e il seguente vettore dei valori $v[2.7, 1.4, 1.5, 0.3]$.

Soluzione.

Dati n oggetti e capacità dello zaino K , l'algoritmo risolve i problemi $P(i, j)$, con $i \in [1, n]$ e $j \in [0, K]$, corrispondenti al valore complessivo massimo ottenibile considerando i primi i oggetti ed uno zaino di capacità j . La matrice di numeri viene usata per memorizzare le soluzioni di tali problemi. La matrice di booleani viene usata per memorizzare se l'oggetto i -esimo fa parte della soluzione al problema $P(i, j)$.

Data l'istanza del problema dello zaino indicata nel testo dell'esercizio, le due matrici vengono riempite nel seguente modo:

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8	j=9	j=10
i=1	0.0	0.0	2.7	2.7	2.7	2.7	2.7	2.7	2.7	2.7	2.7
i=2	0.0	0.0	2.7	2.7	2.7	2.7	2.7	2.7	2.7	4.1	4.1
i=3	0.0	0.0	2.7	2.7	2.7	2.7	2.7	2.7	4.2	4.2	4.2
i=4	0.0	0.0	2.7	2.7	2.7	2.7	3.0	3.0	4.2	4.2	4.2

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8	j=9	j=10
i=1	false	false	true	true	true	true	true	true	true	true	true
i=2	false	false	false	false	false	false	false	false	false	true	true
i=3	false	false	false	false	false	false	false	false	true	true	true
i=4	false	false	false	false	false	false	true	true	false	false	false

4. Un grafo non orientato è *connesso* se esiste un cammino da ogni vertice ad ogni altro vertice. Progettare un algoritmo che dato in input un grafo non orientato G restituisce un booleano: **true** se G è connesso, **false** altrimenti.

Soluzione.

Per verificare se un grafo non orientato è connesso è sufficiente effettuare una visita a partire da un qualsiasi vertice e poi controllare che tutti i vertici del grafo siano stati visitati. Ad esempio, l'Algoritmo 3 riceve in input il grafo non orientato $G = (V, E)$, effettua una visita in ampiezza a partire da un vertice $v \in V$ e poi verifica che tutti i vertici siano stati visitati. Il costo computazionale

di tale algoritmo è quindi la somma del costo della visita, ovvero $O(|V| + |E|)$ assumendo implementazione del grafo tramite liste di adiacenza, più il costo della verifica di marcatura di tutti i vertici, ovvero $O(|V|)$: $O(|V| + |E|) + O(|V|) = O(|V| + |E|)$.

Algorithm 3: VERIFICAConnessione(GRAPH (V, E))

```
// Inizializzazione marcatura
for  $u \in V$  do
   $u.visited \leftarrow false$ 
// Esecuzione della BFS a partire da un vertice  $v \in V$ 
let  $v \in V$ 
QUEUE  $q \leftarrow new\ QUEUE()$ 
 $v.visited \leftarrow true$ 
 $q.enqueue(v)$ 
while not  $q.isEmpty()$  do
   $u \leftarrow q.dequeue()$ 
  for  $w \in u.adjacents()$  do
    if  $w.visited == false$  then
       $w.visited \leftarrow true$ 
       $q.enqueue(w)$ 
// Controllo che tutti i vertici siano stati visitati
for  $u \in V$  do
  if  $u.visited == false$  then
    return false
return true
```
